

PINNSur: Physics-Informed Neural Networks for PDEs on Curved Surfaces

Pranav Jain

University of Southern California

Navami Kairanda

Max Planck Institute for Informatics, SIC

Peter Yichen Chen

University of British Columbia

Oded Stein

University of Southern California
Technion

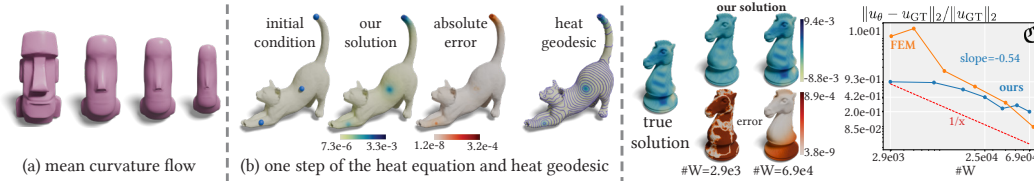


Figure 1: PINNSur solves PDEs on curved surfaces embedded in 3D. We empirically show that the relative ℓ_2 error decreases as the number of tunable weights ($\#W$) of our network increases, indicating convergence of our method on arbitrary surfaces. (a) Our method simulates a mean curvature flow on a mesh. (b) We solve a step of the heat equation given initial heat sources and solve for the heat geodesic distance. (c) We then show how the predicted solution and error change as $\#W$ is increased. The true solution is the cotangent Laplacian solution on a fine mesh.

Abstract

Partial differential equations (PDEs) on surfaces are fundamental to scientific computing and geometry processing. A popular approach to solving PDEs on surfaces is the finite element method (FEM), where the surface is divided into discrete geometric elements (usually triangles). Recently, physics-informed neural networks (PINNs) have emerged as a continuous, mesh-free alternative that does not suffer from FEM’s sensitivity to mesh quality or geometric discretization errors. We present PINNSur, a simple framework for using PINNs on curved surfaces: we train a neural field to approximate the surface’s normals, and then we express surface differential operators using their projection from $\mathbb{R}^3$ onto the surface. Since every orientable manifold has well-defined normals, our method is suitable for all such surfaces, regardless of curvature or topology, enabling many geometry processing applications. Moreover, despite their empirical success in solving PDEs in flat Euclidean domains, PINNs lack convergence guarantees to the true solution of the underlying PDE, and there is limited systematic experimental evidence demonstrating such convergence. This gap restricts their adoption as reliable solvers compared to established methods like FEM, where convergence to the true solution is well understood and theoretically grounded. These surface PDEs are particularly challenging to solve convergently, as one must not only deal with the convergence of the function approximation, but also with the convergence of the geometric approximation of the surface itself. In this work, we empirically investigate the convergence behavior of PINNs for solving surface PDEs by introducing a simple empirical convergence test.

1 Introduction

Partial Differential Equations (PDEs) on curved two-dimensional surfaces $\Omega \subseteq \mathbb{R}^3$ are at the core of many 3D geometry processing applications such as surface editing, animation, physical simulation,

and computer-aided manufacturing. Consequently, accurate, efficient, and easy-to-implement solvers for surface PDEs have become a central component of modern geometry processing pipelines. Curved surfaces pose additional challenges to PDE solving compared to flat two-dimensional regions $\subseteq \mathbb{R}^2$ or $\subseteq \mathbb{R}^3$: One must not only discretize the PDE, but also the surface itself. An extremely common discretization of surfaces has been the triangle mesh, which works well together with the Finite Element Method (FEM) for discretizing PDEs, especially linear (Lagrangian) FEM, which approximates solutions using piecewise linear functions defined on triangular elements. Linear FEM has many advantages, the main one being that it can solve the Poisson equation in a convergent way given certain conditions [Dziuk, 1988, Wardetzky, 2007]. It also has a variety of drawbacks, such as needing meshes with well-behaved triangles and careful refinement.

To address FEM’s drawbacks, Physically Informed Neural Networks (PINNs) have become popular [Karniadakis et al., 2021, Hao et al., 2022], where the function is approximated by a differentiable neural network, and the PDE is solved using *autodifferentiation*. Neural network training methods are then used to find the weights that solve the PDE. PINNs address some of FEM’s shortcomings, such as the dependence on a mesh with high-quality triangles, but are difficult to extend to curved surfaces: The coordinate networks underlying PINNs generally only work on d -dimensional flat subsets of $\mathbb{R}^d$ (i.e., areas in $\mathbb{R}^2$ and volumes in $\mathbb{R}^3$). Recent works [Williamson and Mitra, 2025, Welschinger et al., 2025] have begun to extend PINNs to curved surfaces and work towards integrating PINNs in mainstream geometry processing applications. One line of work formulates PINNs in $\mathbb{R}^3$, but explicitly evaluates a surface differential operator (such as the Laplace-Beltrami operator Δ) using standard formulas from differential geometry. The work of Williamson and Mitra [2025], in particular, does this by parameterizing the surface Ω using a sphere as a parameter domain, and using that parameterization to construct a Laplace-Beltrami operator. There is no explicit discretization of the PDE or the surface geometry into elements; instead, the effective discretization is implicitly controlled by the number of network weights. Thus, unlike FEMs, which provide convergence guarantees to the true solution under consistent discretization (typically as a function of the number of triangular elements n), neural solvers lack analogous guarantees, and systematic studies remain limited.

Building on the ideas of Williamson and Mitra [2025], we introduce a new framework for using PINNs on curved surfaces by completely eliminating the parameter domain, simplifying the network, and extending it to arbitrary manifold topologies. Our key observation is that common surface PDE operators (e.g., Poisson and Helmholtz) do not require a parametric mapping of the surface; normals alone suffice to construct surface differential operators, since one can define the operators in Euclidean space and project out normal components to obtain the surface operators. Our method supports both Dirichlet and Neumann boundary conditions and performs well on many challenging geometries. We then study convergence behavior to the true solution for the proposed surface PDE solver. We introduce a notion for the convergence of PINNs by relating the error to the number of degrees of freedom in the PINN. Our convergence criterion parallels classical convergence criteria in use for FEM, where the error is bounded by an expression containing the number of elements or the element size (both proxies for the total number of degrees of freedom). Analogously, for PINNs, the number of trainable parameters can be interpreted as an effective measure of degrees of freedom. We study both classical PINNs as well as our new PINN for curved surfaces using this criterion, and establish their convergence.

Our contributions are :

1. A simple surface PINN framework called PINNsur for solving PDEs using a normals-only neural field for surface operators.
2. A new empirical convergence criterion for PINNs.
3. An empirical systematic convergence study of PINNs on test functions across varied PDEs, geometries, and boundary conditions.

Our complete code can be accessed here: [code link](#).

2 Related Work

Classical Methods for Surface PDEs. There are many classical methods for solving surface PDEs; we highlight only a few important examples that are used in the geometry processing of curved surfaces. Dziuk [1988] proved an early convergence result for the linear FEM on curved surfaces approximated by triangle meshes, which was later extended by Wardetzky [2007]. These results

generalize known flat FEM theorems [Brenner and Scott, 2008]. As a rule of thumb, such FEM methods have L^2 convergence on the order of $\mathcal{O}(n)$, where n is the number of degrees of freedom. The reliability of linear FEM has led to its popularity in many geometry processing applications such as the computation of minimal surfaces [Pinkall and Polthier, 1993], surface fairing [Desbrun et al., 1999], mesh editing [Sorkine et al., 2004], animation [Baran and Popović, 2007], and many more [Botsch et al., 2010]. Beyond linear FEM, other popular discretization methods include higher-order FEM [Ferguson et al., 2023], discrete exterior calculus [Hirani, 2003], and Monte Carlo methods (which have only recently been extended to surfaces) [Sugimoto et al., 2024] (which converge on the order of $\mathcal{O}(\sqrt{n})$, where n is the number of degrees of freedom).

Physics Informed Neural Networks. PINNs are neural fields that model solutions to differential equations by posing them as optimization problems supervised by physical laws [Karniadakis et al., 2021, Hao et al., 2022, Raissi et al., 2019, Kharazmi et al., 2021]. They offer a flexible alternative to classical discretization-based solvers, especially when data or meshing is challenging. PINNs have been applied across a range of problems, including fluid dynamics [Cai et al., 2021, Jain et al., 2024], volumetric elastodynamics [Rao et al., 2021], Eikonal equations [Smith et al., 2021], and cloth simulation [Kairanda et al., 2024, 2026]. Beyond direct neural solvers, there is growing interest in *neural operators* that learn solution maps for parametric PDEs [Yu et al., 2018, Welschinger et al., 2025]. Early PINN works with coordinate-based MLPs struggled with high-frequency signals and derivatives; Siren [Sitzmann et al., 2020] mitigates this via periodic activations and has enabled broader applications [Chen et al., 2023, Zehnder et al., 2021, Kairanda et al., 2024]. However, Siren and most PINNs are formulated on Euclidean grids rather than surfaces. A concurrent work by [Welschinger et al., 2025] solves PDEs on surfaces via a *neural operator*, but still uses Euclidean finite differences. In contrast, our solver models surface operators directly and solves the PDE on the continuous surface field.

Previous work on PINN convergence includes Shin et al. [2020], which prove that, under certain conditions, PINNs on flat domains in $\mathbb{R}^n$ converge to their solutions, and provide experimental evidence that increasing the amount of training data decreases error. Tang and Fu [2021] investigate the convergence behavior of PINNs on curved surfaces as the number of training samples increases, and they do so for different network depths. Nam et al. [2024] introduces a novel neural Monte Carlo method and investigates its convergence behavior as the network is trained over time. Prior work focuses on training convergence of PINNs, whereas we study convergence with respect to degrees of freedom, where the number of tunable weights plays a role analogous to FEM elements.

Neural Geometry Processing. Many recent methods encode geometric shapes (e.g., surfaces) as neural networks, often as neural fields, enabling geometry processing tasks [Yang et al., 2021, Bednarik et al., 2020, Mehta et al., 2022, Gao et al., 2024, Edavamadathil Sivaram et al., 2024, Fargion and Weber, 2025, Chetan et al., 2025]. A key advantage is that differential geometry quantities like normals and curvatures can be computed via *autodiff*. Morreale et al. [2021] learn explicit surface maps for distortion-minimizing correspondences, and Morreale et al. [2022] disentangles fine detail from global structure, while Aigerman et al. [2022] learns piecewise-linear mesh mappings. Novello et al. [2022, 2023] explore differentiable geometry of level-set surfaces and how to evolve implicit level-set functions, while Yang et al. [2021] demonstrates surface smoothing and sharpening on implicit. Recently, Chang et al. [2025], Williamson and Mitra [2025] solve the Laplace–Beltrami operator to perform spectral analysis. The closest work to ours is Williamson and Mitra [2025]; the key difference is that we avoid a parameterized surface map, since it is not part of the surface PDE operator. We instead use a normal field, which improves accuracy and convergence.

3 Method

Our setting is a (curved) two-dimensional manifold surface $\Omega \subseteq \mathbb{R}^3$. We solve PDEs of the form

$$\mathcal{F}[u(\mathbf{x})] = 0, \quad \mathbf{x} \in \Omega, \quad (1)$$

$$u(\mathbf{x}) = g(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega_D, \quad (2)$$

$$\frac{\partial u}{\partial \nu}(\mathbf{x}) = h(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega_N, \quad (3)$$

where $\mathcal{F}$ is a second-order elliptic differential operator, $\partial\Omega_D$ is the boundary curve with Dirichlet condition and $\partial\Omega_N$ with Neumann condition with ν as the boundary normal. The no-boundary case,

$\partial\Omega_D = \partial\Omega_N = \emptyset$, occurs for some curved surfaces. In this work, we focus on the Poisson and Helmholtz equations because of their wide applications [Kazhdan and Hoppe, 2013, Pinkall and Polthier, 1993].

3.1 Poisson Equation

The (Euclidean) Laplacian $\Delta_{\mathbb{R}^d}$ of a scalar field $u : \mathbb{R}^d \rightarrow \mathbb{R}$ ($d = 2, 3$) is defined as the divergence of the gradient of the field. The Laplace-Beltrami operator, or surface Laplacian, is the generalization of the Laplacian $\Delta_{\mathbb{R}^d} u$ to curved domains Ω , $\Delta_{\Omega} u = \nabla_{\Omega} \cdot \nabla_{\Omega} u$. The surface gradient is the orthographic projection of the Euclidean gradient to the local tangent plane $\nabla_{\Omega} u = \nabla_{\mathbb{R}^d} u - (\nabla_{\mathbb{R}^d} u \cdot \mathbf{n})\mathbf{n}$. Similarly, for a vector field $\mathbf{F} : \mathbb{R}^d \rightarrow \mathbb{R}^d$, the surface divergence is defined as

$$\nabla_{\Omega} \cdot \mathbf{F} = \nabla_{\mathbb{R}^d} \cdot \mathbf{F} - \mathbf{n}^T \mathbf{J}(\mathbf{F}) \mathbf{n}, \text{ where } \mathbf{J}(\mathbf{F}) = \begin{bmatrix} \frac{\partial \mathbf{F}}{\partial x_1} & \frac{\partial \mathbf{F}}{\partial x_2} & \cdots & \frac{\partial \mathbf{F}}{\partial x_d} \end{bmatrix} \quad (4)$$

is the Jacobian of $\mathbf{F}$ [Hubbard and Burke Hubbard, 2015].

The Poisson equation on a curved domain $\Omega \subset \mathbb{R}^d$ for a scalar function $u : \Omega \rightarrow \mathbb{R}$ is given by $\Delta_{\Omega} u = f$, where $f : \Omega \rightarrow \mathbb{R}$ is the source term. It is relatively straightforward to compute the Euclidean Laplacian of a coordinate network by applying *autodifferentiation* to a smooth network $u : \mathbb{R}^d \rightarrow \mathbb{R}$:

$$\Delta_{\mathbb{R}^d} u = \frac{\partial^2 u}{\partial x_1^2} + \frac{\partial^2 u}{\partial x_2^2} + \cdots + \frac{\partial^2 u}{\partial x_d^2}. \quad (5)$$

This simple formula does not apply to the Laplace-Beltrami operator, where we can not simply employ component-wise differentiation. The approach of Williamson and Mitra [2025] is to represent the surface with a parametric neural map $\varphi_{\theta} : \mathbb{S}^2 \rightarrow \Omega$. φ_{θ} is obtained by first computing a discrete parametrization of the surface triangle mesh from a reference sphere, and then training a neural network to approximate that discrete parametrization. While this allows computing normals via the Jacobian of φ_{θ} , it does not handle arbitrary topologies (the topology of the parameter domain must match the topology of Ω). In contrast, our method does not require a parametrization at all. We also represent u as a coordinate network, but instead of a parametrization we directly work with the normal field that is either known exactly, or that we represent with another network that can be trained quickly and used without any autodifferentiation. The resulting formula for the Laplace-Beltrami operator of a coordinate network is:

$$\Delta_{\Omega} u = \nabla_{\mathbb{R}^d} \cdot (\nabla_{\Omega} u) - \mathbf{n}^T \mathbf{J}(\nabla_{\Omega} u) \mathbf{n}. \quad (6)$$

The solution of the actual PDE works as follows. Given a function f and a surface Ω , we train a PINN to learn the function u by defining the loss function $\mathcal{L}_{\text{poisson}}$ as

$$\mathcal{L}_{\text{poisson}} = \|\Delta_{\Omega} u_{\theta} - f\|_2^2. \quad (7)$$

The Laplace-Beltrami operator (Δ_{Ω}) is computed using (6). Please see Fig. 2 for an overview.

3.2 Helmholtz Equation

The Helmholtz equation on a curved domain $\Omega \subset \mathbb{R}^d$ for a scalar function $u : \Omega \rightarrow \mathbb{R}$ is defined as $\Delta_{\Omega} u + k^2 u = f$, where k is the wavenumber and $f : \Omega \rightarrow \mathbb{R}$ is the source term.

We use the same framework as for the Poisson equation, but define the loss function $\mathcal{L}_{\text{helmholtz}}$ as

$$\mathcal{L}_{\text{helmholtz}} = \|\Delta_{\Omega} u_{\theta} + k^2 u_{\theta} - f\|_2^2. \quad (8)$$

3.3 Boundary Conditions

We employ so-called soft boundary constraints [Raissi et al., 2019, Chen et al., 2023], where boundary conditions are enforced by adding penalty terms to the loss function during training.

$$\mathcal{L}_{\text{soft}} = \mathcal{L}_{\text{PDE}} + \lambda_{\text{BC}} \mathcal{L}_{\text{BC}}, \quad (9)$$

where λ_{BC} is a tunable parameter. We choose $\lambda_{\text{BC}} = 100$ to give a high weight to the boundary conditions. Fig. 4 shows that the solution error at the boundary is less dependent on the parameter λ_{BC} , but the error at the boundary decreases as the size of the network increases.

Different boundary conditions necessitate different loss functions $\mathcal{L}_{\text{BC}}$.

Dirichlet Boundary. To solve a PDE with Dirichlet boundary condition $u(\mathbf{x}) = g(\mathbf{x})$, $\mathbf{x} \in \partial\Omega_D$, we add the following term to the loss function.

$$\mathcal{L}_{BC} = \|u_\theta - g\|_2^2. \quad (10)$$

The loss in (10) is minimized exactly when the Dirichlet boundary conditions hold. Since a valid solution to a PDE with Dirichlet boundary conditions exist, there must be a global minimum that both minimizes the main PDE loss ((7), (8)), as well as the additional boundary loss (10).

Neumann Boundary. To enforce Neumann boundary conditions, one needs to compute $\frac{\partial u}{\partial \nu}$. Since we enforce Neumann conditions with a soft loss term involving $\frac{\partial u}{\partial \nu}$, this quantity needs to be differentiable, which we achieve by training another coordinate MLP on the boundary normals ν by computing the cross product between the boundary edge and the surface normal at the boundary. Then, we use this MLP to compute the following term at $\partial\Omega_N$

$$\mathcal{L}_{BC} = \|\nabla_\Omega u_\theta \cdot \nu_{\hat{\theta}} - h\|_2^2. \quad (11)$$

Compatibility. When solving for the Poisson or Helmholtz equations, the compatibility condition is required to ensure the existence of a solution [Evans, 2010]. Existence and uniqueness are guaranteed for both Poisson and Helmholtz when pure Dirichlet boundary conditions are imposed (as long as the boundary condition is compatible with the interior function values). For the Poisson equation $\Delta_\Omega u = f$ and the Helmholtz equation $\Delta_\Omega u + k^2 u = f$ with a Neumann boundary, the solution exists and is unique up to a constant if

$$\int_\Omega f + \int_{\partial\Omega} \frac{\partial u}{\partial \nu} = 0. \quad (12)$$

For surfaces without a boundary, we simply have to ensure $\int_\Omega f = 0$. For all our experiments on surfaces with Neumann or no boundary, we choose the source terms f that ensure homogeneous Neumann boundary ($h(\mathbf{x}) = 0$) and $\int_\Omega f = 0$.

3.4 Network Design & Training

Normal Networks. To compute the Laplace-Beltrami operator (6), we need the surface normals $\mathbf{n}$ at each point on the surface Ω and boundary normals ν for Neumann boundary conditions. We use a *Siren* network [Sitzmann et al., 2020] which is a multilayer perceptron with *sine* activation functions to train both the surface normal field and the boundary normal field. We chose *Siren* because of its ability to approximate high-frequency functions (Fig. 10).

Our surface normal network is $\mathbf{n}_{\hat{\theta}} : \mathbb{R}^d \rightarrow \mathbb{R}^d$, mapping the Euclidean coordinates $\mathbf{x} \in \Omega$ to the normal $\mathbf{n}$ at $\mathbf{x}$ (even though the coordinate network takes points in $\mathbb{R}^3$ as input, we only ever evaluate it on Ω). The last layer of the network normalizes the output to ensure unit normals. Therefore, while training, we focus only on the direction of the normals at each point on Ω . We define the loss function

$$\mathcal{L}_{\text{normals}} = \sum_{i=1}^N (1 - (\mathbf{n}_{\hat{\theta}_i} \cdot \mathbf{n}_{\text{GT}_i}))^2, \quad (13)$$

where N is the number of uniformly sampled points on the surface Ω . In Fig. 12, we illustrate how the PDE approximation error changes as the size of the normal network increases. The error initially decreases and then plateaus, indicating that once the network is sufficiently expressive to approximate the normal field accurately, further increases in its size no longer impact the error. Empirically, we observe that a network with width 64 and depth 5 is adequate for learning the normal field. We train the network for $\approx 1e7$ iterations with a learning rate of $1e^{-4}$. The normal network training converges quickly for simple domains but takes more time for complex geometries.

Our boundary normal network $\nu_{\hat{\theta}} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ takes the Cartesian coordinates $\mathbf{x} \in \partial\Omega_N$ and outputs the boundary normal ν at $\mathbf{x}$ (even though the coordinate network takes points in $\mathbb{R}^d$ as input, we only ever evaluate it on $\partial\Omega_N$). The training hyperparameters for $\nu_{\hat{\theta}}$ are the same as for $\mathbf{n}_{\hat{\theta}}$. We use the same loss function (13) where ν_{GT} is computed using the cross product between the edge and the surface normal.

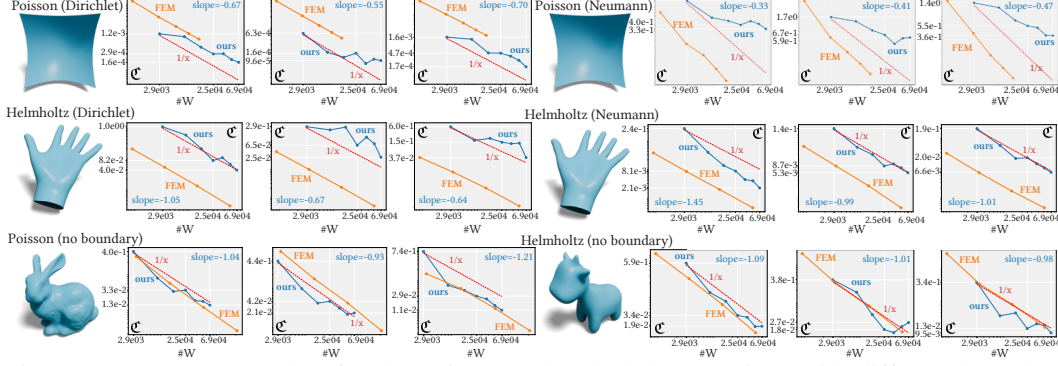


Figure 3: Convergence plots for the Poisson and Helmholtz equations with different boundary conditions for three different functions defined on each surface. For all surfaces and boundary conditions, we show that our method is convergent.

PDE Network u_θ . Once we have the surface normals (and boundary normals) we compute the Laplace-Beltrami operator using (6). Note that since the normals are computed using differentiable neural networks $\mathbf{n}_\theta, \nu_\theta$, we can use *autodiff* to train for u_θ . We train our network u_θ to solve for the Poisson and the Helmholtz equation using (7) and (8) respectively. The boundary loss terms (10) and (11) are added as soft constraints. Fig. 2 shows an overview of our method.

For all experiments, we choose a starting learning rate of $1e^{-3}$ with the *ReduceLROnPlateau* scheduler. We uniformly sample points on the domain and train until u_θ converges and saturates completely. Empirically, we noticed that the training usually converges around $50k$ iterations (Fig. 15). We train all the networks for $150k$ iterations to ensure the network is completely saturated.

To show convergence, we train our PDE network u_θ with depth 3 and vary the width of the network from 30 to 150. In Fig. 11, we show that the choice of varying depth versus width doesn't change the final error. Therefore, without loss of generality, we choose to vary the width in all our experiments. We consider the degrees of freedom to be the total number of tunable weights+biases in the network ($\#W$), which is analogous to the number of vertices in a mesh for FEM. Please see Section 4 for further details.

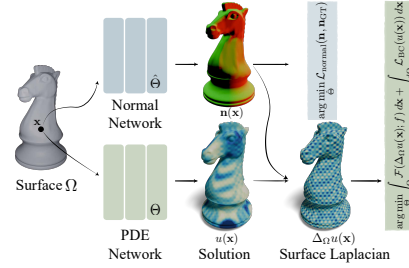


Figure 2: **Overview.** To solve PDEs on a curved surface, we first train a normal network to approximate the surface normals. The PDE network then leverages these normals to compute the surface Laplacian $\Delta_\Omega u$, with boundary conditions added as soft constraints.

4 Convergence

To quantify the accuracy of a PDE solver, we need a notion of convergence. Convergence describes whether the estimate approaches the true solution with respect to a chosen norm. Here we use the relative ℓ_2 norm and define error as

$$e := \|u_\theta - u_{\text{GT}}\|_2 / \|u_{\text{GT}}\|_2, \quad (14)$$

where u_θ and u_{GT} are our optimized solution and the ground-truth, evaluated respectively on $\mathbf{x} \in \Omega$.

In classical discretization-based solvers such as finite-element schemes, convergence is often expressed in relation to refining the number of elements ($\sim$ degrees of freedom) n of the discretization (or a quantity similar to the element size) [Brenner and Scott, 2008]. Convergence is observed if the error decreases as more elements are added to the discretization. In our method (and more broadly in neural solvers that model continuous fields), there is no discretization in the classical sense (and therefore no direct refinement analog). Instead, the weights of the neural field induce an implicit discretization by limiting the model's expressive capacity to represent the continuous solution. We therefore treat the total number of tunable neural network weights ($\#W$) as degrees of freedom and study convergence under the expectation that the error $e \rightarrow 0$ as $\#W \rightarrow \infty$. We compute $\#W$ as

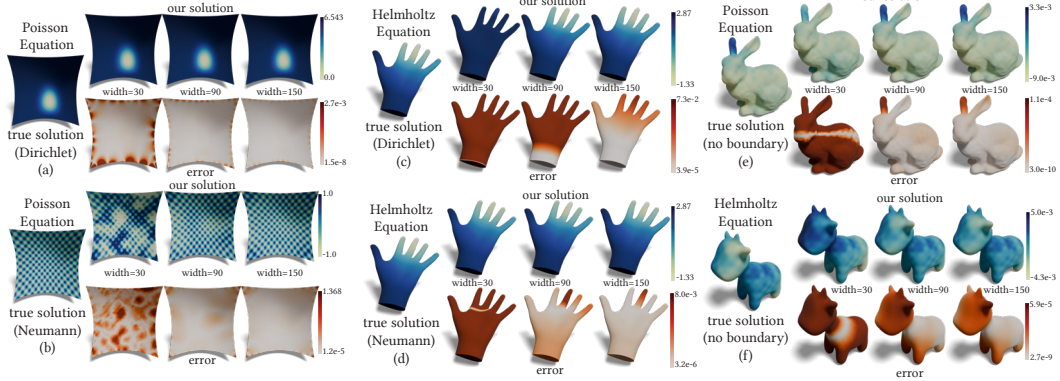


Figure 4: Figure showing the solution and error as network width increases for Poisson and Helmholtz equations under various boundary conditions. We observe that error decreases with increasing width.

the number of parameters of the *Siren* network that models the PDE solution u_θ . Since *Siren* is a coordinate-based MLP, $\#W$ is determined by the depth and width of the MLP. We exclude the normal network $\mathbf{n}_\theta$ from this count, since it is pretrained and unaffected by the PDE. For u_{GT} , we use analytical solutions when available; otherwise, we use a high-resolution FEM solution as GT (cotangent Laplacian [Meyer et al., 2002] on the surface Ω).

With the above setup, we define the following criterion: if the error e in (14) systematically decreases as the number of tunable weights increases, the method converges. To compute this, we fit a line via least-squares linear regression of $\log e$ on $\log(\#W)$ and obtain the slope m and Pearson correlation coefficient r . An experiment is deemed to have converged (indicated by $\mathfrak{C}$ in figures) if $m < -0.3$ and $r < -0.5$, indicating a sufficiently steep decay and a reasonably linear trend. We emphasize that the chosen convergence heuristic is purely empirical, chosen to promote a stable and monotonic decrease in approximation error as the degrees of freedom increase, while mitigating oscillations arising from noise. The use of a lower convergence rate is consistent with previous works in certain areas of numerical analysis, such as Li and Liu [2018], Apel et al. [2018].

5 Results

5.1 Solving PDEs

We solve the Poisson equation on a wide variety of domains: flat surfaces $\subseteq \mathbb{R}^2$, where our method simplifies to a standard PINN (Fig. 7), curved surfaces with boundary where exact normals and boundary normals are known (Fig. 4(a, b)), and arbitrary manifold surfaces given as triangle meshes, where we train a neural network to have a smooth normal field that approximates the piecewise constant mesh normals (Fig. 4(e)). Beyond the Poisson equation, our method also solves the Helmholtz (Fig. 4(c, d, f)) and heat equations (Fig. 1(b)). In Fig. 8, we demonstrate our ability to solve the Helmholtz equation on a challenging intricate curved mesh with fine geometric detail. The details of the experimental setup are provided in Section B.

5.2 Convergence Experiments

Using our convergence criterion established in Section 4, we evaluate and observe the convergence of our method for both Poisson and Helmholtz equations on a wide variety of surfaces with different boundary conditions using different source terms (see Fig. 3). We compare convergence rates with Kharazmi et al. [2021], Yu et al. [2018], Kairanda et al. [2026] and FEM on a flat 2D domain, matching FEM convergence rate (Fig. 7). On curved surfaces, we compare against FEM and a parameterized PINN [Williamson and Mitra, 2025] (see Section 5.4 for more details). The runtimes for each convergence experiment in Fig. 4 can be found in Section C of the supplemental.

5.3 Ablation Study

To justify our parameter choices, we perform ablations. In all our experiments, we vary the network width while keeping the depth constant. Fig. 11 shows that the choice of altering depth instead of width gives similar results. In Fig. 10, we show that the *Siren* network is much better at learning

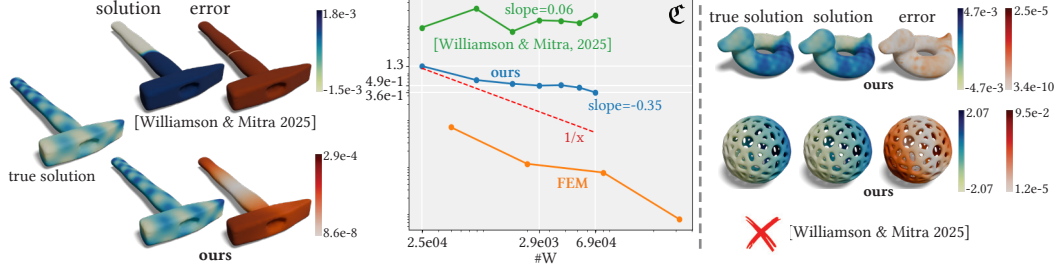


Figure 5: **Comparison with Williamson and Mitra [2025]**. By directly training on surface normals, our method achieves a lower PDE solution error. Moreover, our approach applies to arbitrary topologies, whereas Williamson and Mitra [2025] is limited to surfaces homeomorphic to a sphere.

high-frequency functions compared to using *sigmoid* or *tanh* activation functions. In Fig. 12, we research the dependence of the error on the normal network size. The error initially decreases and then remains constant, indicating that once the normal network is deep enough, only the main network weights matter.

5.4 Comparison with Baselines

Our PINN exhibits a significantly reduced sensitivity to mesh quality, unlike FEM (Fig. 9). Classical PINNs such as Kharazmi et al. [2021], Yu et al. [2018] cannot directly solve surface PDEs, so we compare FEM, Kairanda et al. [2026], Kharazmi et al. [2021], Yu et al. [2018], and our method on $\mathbb{R}^2$. Our approach exhibits convergence behavior similar to FEM for comparable degrees of freedom (Fig. 7), while Kharazmi et al. [2021], Yu et al. [2018] do not converge under the network refinement strategy used here. We attribute this difference to the use of Siren, which better captures the high-frequency solution u compared to the *tanh* of Kharazmi et al. [2021] and $\max(0, x^2)$ of Yu et al. [2018]. Although Kairanda et al. [2026] demonstrates convergence with increasing resolution, improvements eventually saturate due to interpolation assumptions.

On surfaces $\Omega \subset \mathbb{R}^3$, we compare against a parameterization-based approach similar to Williamson and Mitra [2025] (Fig. 5). We speculate that their weaker convergence stems from challenges in learning parameterizations and differentiation errors, both avoided by our method. Note that we support arbitrary manifold topologies, unlike Williamson and Mitra [2025]. We also compare with Sugimoto et al. [2024], a Monte Carlo method limited to Dirichlet problems with simple source terms, while our approach supports both Dirichlet and Neumann boundaries. (Fig. 13)

5.5 Geometry Processing Applications

Since our method works reliably on a wide variety of curved surfaces, it can be used directly for a wide range of geometry processing applications where FEM would traditionally be used.

Minimal Surfaces. A surface $\Omega \subset \mathbb{R}^3$ is minimal if its mean curvature is zero everywhere. The mean curvature relates to the Laplace–Beltrami operator via $\Delta_\Omega = -2H\mathbf{n}$, where H is the mean curvature and $\mathbf{n}$ the surface normal. To compute a minimal surface from a cylinder, we solve three Laplace equations: $\Delta_\Omega u = 0$ with Dirichlet boundary conditions for each coordinate (x, y, z) . The resulting solutions define the new vertex positions. This recovers the cylinder’s minimal surface, the catenoid (Fig. 6(a)).

Harmonic Interpolation. In Fig. 6(b), we show that PINNs can be used to compute the harmonic interpolation of a surface given the Dirichlet boundary condition by solving the Laplace equation

$$\Delta_\Omega u = 0, \quad u(\mathbf{x}) = g(\mathbf{x}), \mathbf{x} \in \partial\Omega_D. \quad (15)$$

Here, our method’s ability to handle all domain topologies particularly shines, since interpolation examples can have many different disconnected boundaries.

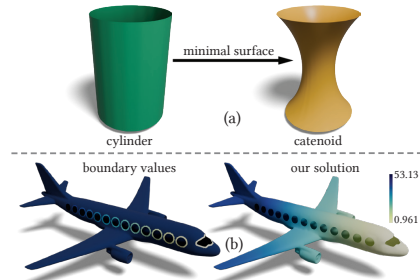


Figure 6: (a) Result of the minimal surface computed for the cylinder. (b) Result of harmonic interpolation in the interior of the plane given Dirichlet boundary values on the windows of the plane.

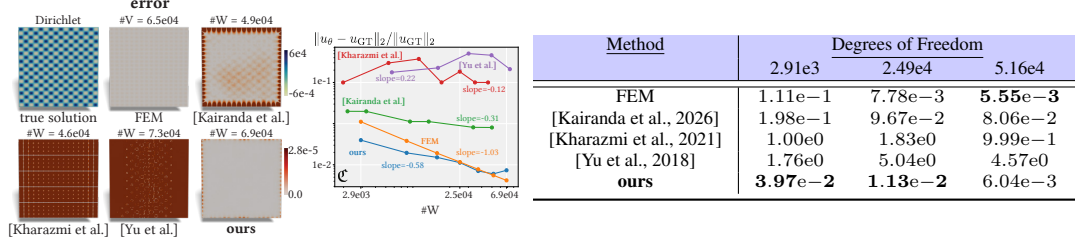


Figure 7: **Poisson solve on a Euclidean domain.** A comparison with FEM, Kairanda et al. [2026], Kharazmi et al. [2021], Yu et al. [2018] for similar degrees of freedom shows that the relative ℓ_2 error and convergence for ours match more closely with FEM.

Heat Equation. Laplacians can also be used to solve the heat equation. The implicit Euler finite difference scheme in time for the heat equation $\frac{\partial u}{\partial t} = \Delta_\Omega u$, results in a screened Poisson equation

$$\Delta_\Omega u^{t+1} - h^{-1}u^{t+1} = -h^{-1}u^t, \quad (16)$$

where h is the time-step size. PINNsur solves the heat equation on intricate curved surfaces by training for each time step. One-step solution of the heat equation given initial heat sources is shown in Fig. 1(b). We can also compute heat-based geodesic distances on a surface using the method of Crane et al. [2017] as shown in Fig. 1(b).

Mean Curvature Flow. Using the heat equation, we can also simulate a mean curvature flow (Fig. 1(a)) by solving (16) three times for $u^t = x, y, z$ where x, y, z are the mesh vertex coordinates. The solution u^{t+1} forms the new vertex coordinates.

6 Limitations

While we show convergence of our method on a range of surfaces and boundary conditions, our method fails if the surface has a discontinuous normal field. For example, as shown in Fig. 14, the boundary normals are discontinuous near the corners, and hence the model fails to converge when solving a Poisson equation with Neumann boundary. Our method also fails to converge if the solution u has a high frequency or if the range of the function is too high. Note that our method can sometimes have a higher error than FEM and incur greater computational cost due to training, similar to other PINNs, but does not require high-quality meshes like FEM.

7 Conclusion & Discussion

We introduced PINNsur for solving second-order PDEs like the Poisson or Helmholtz equation on curved surfaces. PINNsur works on manifold surfaces of arbitrary topology (unlike prior PINNs, which either do not support surfaces at all or are limited to genus-0 surfaces), and offers better convergence behavior than state-of-the-art PINNs. We also introduce an empirical convergence criterion for PINNs, generalizing the principle used for determining the convergence of FEM: the error should decrease as the number of degrees of freedom of the method increases. We use this criterion to study the convergence of existing PINNs.

As promising future work, we plan to add a theoretical convergence analysis to our empirical convergence study of PINNs. For classical PINNs consisting of an MLP accepting a coordinate input, we will follow existing analysis on the expressiveness of neural networks [Lu et al., 2017] to bound the approximation error of a PINN with regard to its width. This expressiveness analysis will be combined with existing PINN convergence analysis that currently lacks degrees of freedom analysis (see Section 2) to compute an error bound for the solution of a PDE that depends on the width and depth of the MLP in our PINN. We will also combine this approach with well-known neural network scaling laws for PINNs to better understand the effects of other training parameters [Lai et al., 2026]. While our current convergence analysis primarily focuses on PDE network width, with preliminary analysis on other hyperparameters such as PDE network depth and normal network, we are also excited to explore more sophisticated refinement strategies in future work.

8 Acknowledgements

This research is generously supported by a gift from Adobe Inc and a grant from the National Science Foundation (award #2335493).

We acknowledge and thank the authors of the 3D assets used in this paper. These models include Bob [Crane, 2026], Bunny [The Stanford 3D Scanning Repository, 2021], Cat [billyd, 2023], Dziuk [Sugimoto et al., 2024], Hand [Max Planck Society e.V via SMPL, 2021], High-genus Sphere [Jan, 2026], lion [glasgowmuseums, 2017], Moai [Zames1992, 2024], Mushroom [Holinaty, 2023], Plane [YahooJAPAN, 2019], Spot [Crane, 2019], [Spognetta, 2023], Springer [TimEdwards, 2023].

References

- N. Aigerman, K. Gupta, V. G. Kim, S. Chaudhuri, J. Saito, and T. Groueix. Neural jacobian fields: learning intrinsic mappings of arbitrary meshes. *ACM Transactions on Graphics*, 2022.
- T. Apel, J. Pfeifferer, S. Rogovs, and M. Winkler. l^∞ -error estimates for neumann boundary value problems on graded meshes, 2018. URL <https://arxiv.org/abs/1804.10904>.
- I. Baran and J. Popović. Automatic rigging and animation of 3d characters. *ACM Trans. Graph.*, 26(3):72–es, 2007.
- J. Bednarik, S. Parashar, E. Gundogdu, M. Salzmann, and P. Fua. Shape reconstruction by learning differentiable surface representations. In *Computer Vision and Pattern Recognition (CVPR)*, 2020.
- billyd. Cat, 2023. Downloaded modified version from [odedstein-meshes github.com/odedstein-meshes/tree/master/objects/cat](https://github.com/odedstein-meshes/odedstein-meshes/tree/master/objects/cat), originally from [thingiverse.com/thing:1565405](https://www.thingiverse.com/thing/1565405). Asset licensed under CC BY 4.0.
- M. Botsch, L. Kobbelt, M. Pauly, P. Alliez, and B. Lévy. *Polygon Mesh Processing*. Taylor & Francis, 2010.
- S. C. Brenner and L. R. Scott. *The Mathematical Theory of Finite Element Methods*. Springer, 2008.
- S. Cai, Z. Mao, Z. Wang, M. Yin, and G. E. Karniadakis. Physics-informed neural networks (pinns) for fluid mechanics: a review. *Acta Mechanica Sinica*, 2021.
- Y. Chang, O. Benckekroun, M. M. Chiaramonte, P. Y. Chen, and E. Grinspun. Shape space spectra. *ACM Transactions on Graphics (TOG)*, 2025.
- H. Chen, R. Wu, E. Grinspun, C. Zheng, and P. Y. Chen. Implicit neural spatial representations for time-dependent pdes. In *International Conference on Machine Learning (ICML)*, 2023.
- A. Chetan, G. Yang, Z. Wang, S. Marschner, and B. Hariharan. Accurate differential operators for hybrid neural fields. In *Computer Vision and Pattern Recognition (CVPR)*, 2025.
- K. Crane. Spot, 2019. Downloaded modified version from [odedstein-meshes github.com/odedstein-meshes/tree/master/objects/spot](https://github.com/odedstein-meshes/odedstein-meshes/tree/master/objects/spot), originally from [cs.cmu.edu/~kmc Crane/Projects/ModelRepository/](https://www.cs.cmu.edu/~kmc Crane/Projects/ModelRepository/).
- K. Crane. Bob, 2026. <https://www.cs.cmu.edu/~kmc Crane/Projects/ModelRepository/>. Asset licensed under CC0 1.0 Universal.
- K. Crane, C. Weischedel, and M. Wardetzky. The heat method for distance computation. *Commun. ACM*, 60(11):90–99, Oct. 2017. ISSN 0001-0782. doi: 10.1145/3131280. URL <https://doi.org/10.1145/3131280>.
- M. Desbrun, M. Meyer, P. Schröder, and A. H. Barr. Implicit fairing of irregular meshes using diffusion and curvature flow. In *Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques*, page 317–324, 1999.
- G. Dziuk. *Finite Elements for the Beltrami operator on arbitrary surfaces*, pages 142–155. Springer Berlin Heidelberg, Berlin, Heidelberg, 1988.

- V. Edavamadathil Sivaram, T.-M. Li, and R. Ramamoorthi. Neural geometry fields for meshes. In *ACM SIGGRAPH 2024 Conference Papers*, 2024.
- L. Evans. *Partial Differential Equations*. American Mathematical Society, Mar. 2010. ISBN 9781470411442. doi: 10.1090/gsm/019. URL <http://dx.doi.org/10.1090/gsm/019>.
- G. Fargion and O. Weber. Learning Metric Fields for Fast Low-Distortion Mesh Parameterizations. *Computer Graphics Forum*, 2025.
- Z. Ferguson, P. Jain, D. Zorin, T. Schneider, and D. Panozzo. High-order incremental potential contact for elastodynamic simulation on curved meshes. In *ACM SIGGRAPH 2023 Conference Proceedings*, 2023.
- A. Gao, M. Chu, M. Kapadia, M. C. Lin, and H.-T. D. Liu. An intrinsic vector heat network. In *International Conference on Machine Learning (ICML)*, 2024.
- glasgowmuseums. Wooden head from above a fireplace (2), 2017. <https://sketchfab.com/3d-models/wooden-head-from-above-a-fireplace-2-aacc5088078743b39844b9331bb41eab>. Asset licensed under CC BY-NC.
- Z. Hao, S. Liu, Y. Zhang, C. Ying, Y. Feng, H. Su, and J. Zhu. Physics-informed machine learning: A survey on problems, methods and applications. *arXiv preprint arXiv:2211.08064*, 2022.
- A. Hirani. *Discrete Exterior Calculus*. PhD thesis, California Institute of Technology, 2003.
- J. Holinaty. Mushroom, 2023. Downloaded from odedstein-meshes github.com/odedstein/meshes/tree/master/objects/mushroom. Asset licensed under CC BY-NC 4.0.
- J. H. Hubbard and B. Burke Hubbard. *Vector Calculus, Linear Algebra, and Differential Forms: A Unified Approach (5th edition)*. Matrix Editions, Sept. 2015. URL <https://hal.science/hal-01297648>.
- P. Jain, Z. Qu, P. Y. Chen, and O. Stein. Neural monte carlo fluid simulation. In *ACM SIGGRAPH 2024 Conference Papers*, 2024.
- C. Jan. high-genus 3d shape, 2026. <https://www.craftsmanspace.com/free-3d-models/math-art-polyhedron-structures.html>. Asset licensed under CC BY-SA 4.0.
- N. Kairanda, M. Habermann, C. Theobalt, and V. Golyanik. Neuralclothsim: Neural deformation fields meet the thin shell theory. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- N. Kairanda, S. Naik, M. Habermann, A. Sharma, C. Theobalt, and V. Golyanik. ∂^∞ -grid: A neural differential equation solver with differentiable feature grids. In *International Conference on Learning Representations (ICLR)*, 2026.
- G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang. Physics-informed machine learning. *Nature Reviews Physics*, 2021.
- M. Kazhdan and H. Hoppe. Screened poisson surface reconstruction. *ACM Transactions on Graphics*, 32(3):1–13, June 2013. ISSN 1557-7368. doi: 10.1145/2487228.2487237. URL <http://dx.doi.org/10.1145/2487228.2487237>.
- E. Kharazmi, Z. Zhang, and G. E. Karniadakis. hp-vpinns: Variational physics-informed neural networks with domain decomposition. *Computer Methods in Applied Mechanics and Engineering*, 374:113547, Feb. 2021. ISSN 0045-7825. doi: 10.1016/j.cma.2020.113547. URL <http://dx.doi.org/10.1016/j.cma.2020.113547>.
- J. Lai, S. Wang, C. Wang, and Y. Lu. Unveiling the scaling law of PINNs under non-euclidean geometry, 2026. URL <https://openreview.net/forum?id=7NS4qtPM2Q>.
- Q. Li and X. Liu. Explicit finite element error estimates for nonhomogeneous neumann problems. *Applications of Mathematics*, 63(3):367–379, 2018. doi: 10.21136/am.2018.0095-18. URL <http://dx.doi.org/10.21136/AM.2018.0095-18>.

- Z. Lu, H. Pu, F. Wang, Z. Hu, and L. Wang. The expressive power of neural networks: a view from the width. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 6232–6240, 2017.
- Max Planck Society e.V via SMPL. Hand, 2021. Downloaded modified version from odedstein-meshes github.com/odedstein/meshes/tree/master/objects/hand, adapted from M. Loper, N. Mahmood, J. Romero, G. Pons-Moll, M. Black "SMPL" (SIGGRAPH Asia 2015). Asset licensed under CC BY 4.0.
- I. Mehta, M. Chandraker, and R. Ramamoorthi. A level set theory for neural implicit evolution under explicit flows. In *European Conference on Computer Vision (ECCV)*, 2022.
- M. Meyer, M. Desbrun, P. Schröder, and A. H. Barr. Discrete differential-geometry operators for triangulated 2-manifolds. In *International Workshop on Visualization and Mathematics*, 2002. URL <https://api.semanticscholar.org/CorpusID:11850545>.
- L. Morreale, N. Aigerman, V. G. Kim, and N. J. Mitra. Neural surface maps. In *Computer Vision and Pattern Recognition (CVPR)*, 2021.
- L. Morreale, N. Aigerman, P. Guerrero, V. G. Kim, and N. J. Mitra. Neural convolutional surfaces. In *Computer Vision and Pattern Recognition (CVPR)*, 2022.
- H. C. Nam, J. Berner, and A. Anandkumar. Solving poisson equations using neural walk-on-spheres. *International Conference on Machine Learning (ICML)*, 2024.
- T. Novello, G. Schardong, L. Schirmer, V. Da Silva, H. Lopes, and L. Velho. Exploring differential geometry in neural implicits. *Computers & Graphics*, 2022.
- T. Novello, V. da Silva, G. Schardong, L. Schirmer, H. Lopes, and L. Velho. Neural implicit surface evolution. In *International Conference on Computer Vision (ICCV)*, 2023.
- U. Pinkall and K. Polthier. Computing discrete minimal surfaces and their conjugates. *Experiment. Math*, 2:15–36, 1993.
- M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 2019.
- C. Rao, H. Sun, and Y. Liu. Physics-informed deep learning for computational elastodynamics without labeled data. *Journal of Engineering Mechanics*, 2021.
- Y. Shin, J. Darbon, and G. E. Karniadakis. On the convergence of physics informed neural networks for linear second-order elliptic and parabolic type pdes. *arXiv preprint arXiv:2004.01806*, 2020.
- V. Sitzmann, J. Martel, A. Bergman, D. Lindell, and G. Wetzstein. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- J. D. Smith, K. Azizzadenesheli, and Z. E. Ross. Eikonet: Solving the eikonal equation with deep neural networks. *IEEE Transactions on Geoscience and Remote Sensing*, 2021.
- O. Sorkine, D. Cohen-Or, Y. Lipman, M. Alexa, C. Rössl, and H.-P. Seidel. Laplacian surface editing. In *Proceedings of the 2004 Eurographics/ACM SIGGRAPH Symposium on Geometry Processing*, page 175–184, 2004.
- A. Spognetta. Hammer, 2023. Downloaded modified version from odedstein-meshes github.com/odedstein/meshes/tree/master/objects/hammer, originally from sketchfab.com/3d-models/hammer-rawscan-01e5b5acad9843818529d15663df5b9e. Asset licensed under CC BY 4.0.
- R. Sugimoto, N. King, T. Hachisuka, and C. Batty. Projected walk on spheres: A monte carlo closest point method for surface pdes. In *SIGGRAPH Asia 2024 Conference Papers*, 2024.
- Z. Tang and Z. Fu. Physics-informed neural networks for elliptic partial differential equations on 3d manifolds. *arXiv preprint arXiv:2103.02811*, 2021.

- The Stanford 3D Scanning Repository. Bunny, 2021. Downloaded modified version from odedstein-meshes github.com/odedstein/meshes/tree/master/objects/bunny. Original mesh at graphics.stanford.edu/data/3Dscanrep.
- TimEdwards. Springer, 2023. Downloaded modified version from odedstein-meshes github.com/odedstein/meshes/tree/master/objects/springer, originally from [thingiverse.com/thing:335658](https://thingiverse.com/thing/335658). Asset licensed under GPLv2.
- M. Wardetzky. *Discrete Differential Operators on Polyhedral Surfaces – Convergence and Approximation*. PhD thesis, Freie Universität Berlin, 2007.
- L. Welschinger, Y. Liu, Z. Wang, and N. Mitra. Learning to solve pdes on neural shape representations. *arXiv preprint arXiv:2512.21311*, 2025.
- R. Williamson and N. J. Mitra. Neural geometry processing via spherical neural surfaces. *Compututer Graphics Forum*, 2025.
- YahooJAPAN. Plane, 2019. Downloaded modified version from odedstein-meshes github.com/odedstein/meshes/tree/master/objects/plane, originally from [thingiverse.com/thing:182252](https://thingiverse.com/thing/182252). Asset licensed under CC BY 3.0.
- G. Yang, S. Belongie, B. Hariharan, and V. Koltun. Geometry processing with neural fields. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- B. Yu et al. The deep ritz method: a deep learning-based numerical algorithm for solving variational problems. *Communications in Mathematics and Statistics*, 2018.
- Zames1992. Moai, 2024. <https://sketchfab.com/3d-models/moai-free-bf7f8fbcdcd7408f9f95567bf9cde054>. Asset licensed under CC BY.
- J. Zehnder, Y. Li, S. Coros, and B. Thomaszewski. Ntopo: Mesh-free topology optimization using implicit neural representations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Physics-Informed Neural Networks for PDEs on Curved Surfaces

—Appendices—

A	Further Experiments	14
A.1	Intricate Mesh	14
A.2	FEM Solution Dependence on Mesh Quality	14
A.3	Further Ablations	14
A.4	Comparison with [Sugimoto et al., 2024]	16
A.5	Limitation Examples	16
B	Experimental Setup	17
C	Computational Timings	17
C.1	Training Convergence Plots	18
D	Source Code Access	18

A Further Experiments

A.1 Intricate Mesh

In Fig. 8, we show the capability of our method to solve the Helmholtz equation on a mesh with high curvature and fine details underscoring the effectiveness of our method.

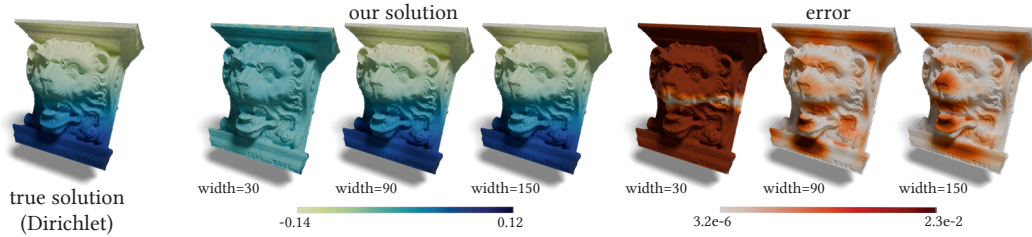


Figure 8: Our method can effectively solve PDEs even on mesh with intricate details.

A.2 FEM Solution Dependence on Mesh Quality

In Fig. 9, we solve the Poisson equation using FEM on meshes of varying quality and compare it with our PINN framework, which samples points directly from the exact surface. As mesh quality deteriorates, FEM accuracy degrades due to discretization errors, whereas our method remains robust, relying only on the ability to sample surface points rather than mesh quality.

A.3 Further Ablations

In Fig. 10, we show that the use of the *Siren* network with *sine* activation function results in a much lower error compared to using *tanh* or *sigmoid* activation functions. This experiment justifies our use of *Siren* network both for the normal and the PDE network.

Since we use the number of tunable weights $\#W$ as our degrees of freedom, there are two ways to increase $\#W$ for an MLP – by increasing the depth or by increasing the width of the MLP. In Fig. 11,

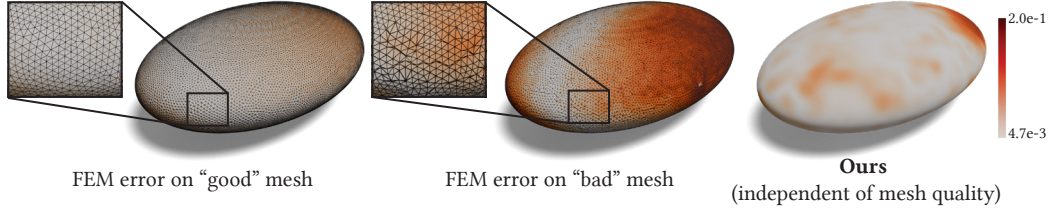


Figure 9: While the FEM error is dependent on the mesh quality – error increases with mesh degradation, our method is independent of mesh quality.

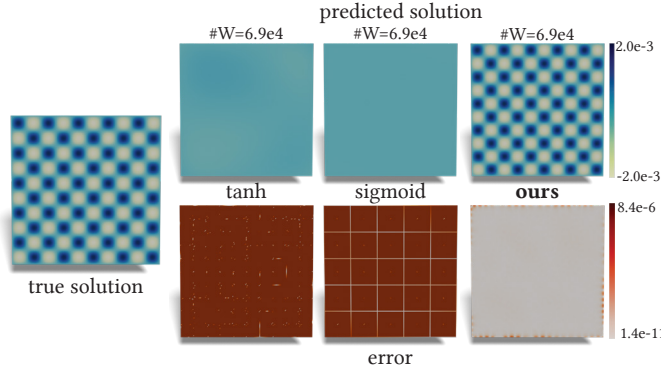


Figure 10: An ablation study where we compare the *Siren* network, which uses *sine* activation function with *Tanh* and *Sigmoid* activation functions on a 2D domain. The figure shows that *Tanh* and *Sigmoid* fail to solve for a high-frequency function.

we show that both choices result in the same error for equal number of tunable weights $\#W$. In all our experiments we choose to increase the width of the network to increase $\#W$.

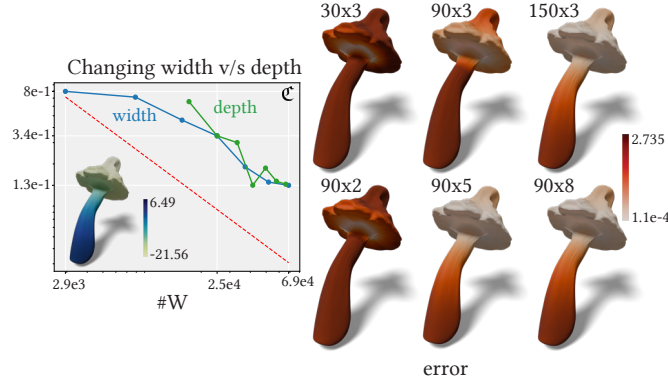


Figure 11: Result of an ablation study where we compare the final error first by changing the width, keeping the depth constant, and vice-versa. The figure shows that the final error in the solution to the PDE doesn't change when the network width is changed, compared to when the depth is changed.

In Fig. 12, we show how the final error behaves as the normal network size is increased. We observe that the error first decreases and then stays constant showing that once the normal network is large enough to accurately learn the surface normals, the size of the normal network doesn't influence the final PDE error.

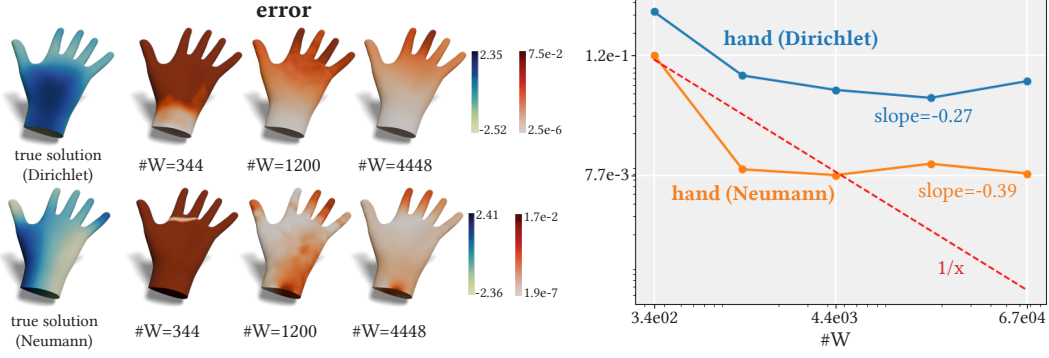


Figure 12: Figure showing how the error varies when the tunable parameters of the normal network are increased keeping the PDE network parameters constant.

A.4 Comparison with [Sugimoto et al., 2024]

In Fig. 13, we compare our results with those of [Sugimoto et al., 2024], a Monte Carlo-based approach. Our method demonstrates lower error when solving the screened Poisson equation on the Dziuk surface.

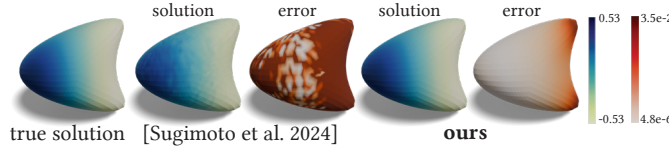


Figure 13: Comparison of our method with [Sugimoto et al., 2024] which is a Monte-Carlo based method. We show that our method achieves a lower error when solving a screened Poisson equation on the Dziuk surface.

A.5 Limitation Examples

Fig. 14 illustrates the limitations of our approach where our network fails to solve for the given PDE. Enforcing Neumann boundary conditions becomes challenging in regions where the normal is discontinuous (e.g., at the corners of the height field). Additionally, the method breaks down when the solution u exhibits large variance.

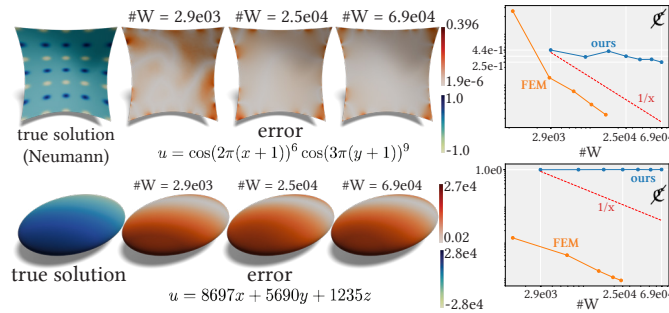


Figure 14: Figure showing the limitations of our work. Enforcing Neumann boundary is difficult at regions where the normal is discontinuous (for ex., at the corners of the heightfield). The method also fails if the solution u has a huge variance.

B Experimental Setup

We set up experiments by first defining a pair of functions u and f on the given surface Ω that ensure homogeneous boundary conditions and the compatibility condition (12). We define analytic u , f from a family of polynomial and trigonometric functions for surfaces whose normal field is known exactly. For surfaces with boundary and no analytic form of the normal field, we use the eigenfunctions of the Laplace-Beltrami operator ϕ as the function u . This gives

$$\Delta_{\Omega}\phi_i = \lambda_i\phi_i = f, \quad (17)$$

where ϕ_i and λ_i are the i^{th} eigenfunction and eigenvalue. Similarly, for the Helmholtz equation

$$\Delta_{\Omega}\phi_i + k^2\phi_i = (\lambda_i + k^2)\phi_i = f. \quad (18)$$

For surface meshes with no boundary and no analytic normal field, we define the source terms f from a family of polynomial and trigonometric functions and compute u using the cotangent Laplacian on a fine mesh. To ensure compatibility, we compute $\int_{\Omega} f dA$ using the mass matrix of the triangle mesh and subtract it from f .

After defining the pairs u and f , we set up our network with different widths and train each of them until complete saturation. Once every network is converged, we compute the relative ℓ_2 error and plot it on a log-log plot to judge convergence as discussed in Section 4. Note that for surfaces where the true analytical solution is not known, we compute u_{GT} by solving

$$-Lu_{GT} = Mf \quad (19)$$

for the Poisson equation, where L and M are the cotangent Laplacian and the mass matrix of a fine triangular mesh, respectively. Similarly, for the Helmholtz equation, we solve

$$(-L + k^2I)u_{GT} = Mf. \quad (20)$$

Given an analytical solution (or FEM solution with cotangent Laplacian) on a surface Ω , we study the convergence patterns by training a PDE network u_{θ} of varying widths keeping the depth constant, and we plot the log of relative ℓ_2 error with respect to the network parameters $\#W$ for each network after it's fully trained (Fig. 3).

C Computational Timings

We ran all the experiments on an Intel i7 5.2GHz with a NVIDIA RTX 4080 GPU. We report the timings for each example shown in Fig. 4 (Figure showing our solution and the error as the network width sizes are increased for Poisson and Helmholtz equations with different boundary conditions) below. Note that we train each model for 150000 iterations to ensure saturation but most models converge way before 150000 iterations. Therefore, we report timing until convergence and also for full 150000 iterations.

The computational time for solving the Poisson equation with Dirichlet boundary on the heightfield example (Fig. 4a) is as follows:

Width, Depth	Runtime (150000 iters)	Iterations to Convergence	Runtime to Convergence
30, 3	26m 16s	51039	8m 56s
90, 3	29m 22s	69488	13m 42s
150, 3	37m 58s	33319	8m 26s

The computational time for solving the Helmholtz equation with Neumann boundary on the heightfield example (Fig. 4b) is as follows:

Width, Depth	Runtime (150000 iters)	Iterations to Convergence	Runtime to Convergence
30, 3	29m 49s	61109	11m 48s
90, 3	35m 20s	54123	12m 54s
150, 3	46m 24s	41125	12m 40s

The computational time for solving the Helmholtz equation with Dirichlet boundary on the hand example (Fig. 4c) is as follows:

Width, Depth	Runtime (150000 iters)	Iterations to Convergence	Runtime to Convergence
30, 3	19m 47s	38630	5m 12s
90, 3	24m 50s	33408	5m 46s
150, 3	32m 40s	34051	7m 32s

The computational time for solving the Helmholtz equation with Neumann boundary on the hand example (Fig. 4d) are as follows:

Width, Depth	Runtime (150000 iters)	Iterations to Convergence	Runtime to Convergence
30, 3	27m	35826	6m 46s
90, 3	34m	35995	8m 21s
150, 3	41m 46s	26786	7m 38s

The computational time for solving the Poisson equation on the bunny example (Fig. 4e) are as follows:

Width, Depth	Runtime (150000 iters)	Iterations to Convergence	Runtime to Convergence
30, 3	17m 23s	46481	5m 29s
90, 3	22m 47s	41163	6m 11s
150, 3	30m	29613	5m 53s

The computational time for solving the Helmholtz equation on the spot example (Fig. 4f) are as follows:

Width, Depth	Runtime (150000 iters)	Iterations to Convergence	Runtime to Convergence
30, 3	18m 8s	48626	6m
90, 3	23m 10s	29557	4m 39s
150, 3	29m	32337	6m 31s

For normal and boundary normal networks, we train a fixed size network (width 64 and depth 5) for $1e7$ iterations for all examples. The normals are computed either analytically (if known) or using the mesh normals. The computational time for training the normal network is about $2h40m$ (1050 iterations per second).

C.1 Training Convergence Plots

Fig. 15 shows the training loss for different values of $\#W$ on the "bunny" model shown in Fig. 4e. The network typically converges around $50k$ iterations. However, we train for $150k$ iterations to ensure full saturation.

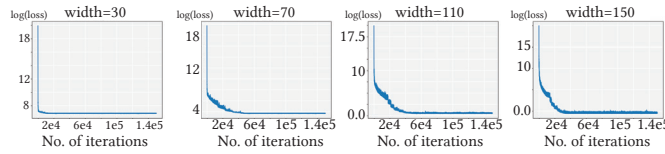


Figure 15: Figure showing the training loss for different $\#W$. The network usually converges around $50k$ iterations, but we train for $150k$ iterations to ensure complete saturation of the network.

D Source Code Access

Our complete code can be accessed here: [code link](#). The README.md contains information on setting up the environment and running scripts to reproduce the experimental results reported in the paper.